

Reviewer Pack — Eigenvalue Threshold in SOS Moment Matrices for Max-CUT Appr...

Entry 03dda0d662e2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 07:24:15 UTC

Eigenvalue Threshold in SOS Moment Matrices for Max-CUT Approximation

Entry ID: 03dda0d662e2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 07:24:15 UTC

1. Conjecture

Field A (mathematical branch): REAL_ALGEBRAIC_GEOMETRY **Field B** (complexity object): COMMUNICATION_COMPLEXITY

Statement:

For any degree- d SOS relaxation of max-CUT on n -vertex graphs, if the moment matrix M has all eigenvalues λ_i satisfying $|\lambda_i| \leq C * \sqrt{n/d^3}$ for some universal constant C , then the relaxation cannot approximate max-CUT with ratio better than $0.878 - \varepsilon$ for any $\varepsilon > 0$. This eigenvalue bound is preserved under polynomial-time reductions between max-CUT instances.

Rationale (proposer's reasoning):

The conjecture ties SOS moment matrix spectral properties to approximation ratios, leveraging real algebraic geometry's Positivstellensatz and communication complexity's reduction invariance. If validated, it would establish a quantitative barrier for SOS-based max-CUT approximations, aligning with Bouland-Kothari's planted clique lower bounds while avoiding algebrization barriers.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1ab3936f60d12eb3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        factor = Fraction(A[i][i], 1)
        for j in range(n):
            A[i][j] /= factor
        for k in range(n):
            if k != i:
                factor = Fraction(A[k][i], 1)
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    d = random.randint(2, 10)
    C = Fraction(1, 1) # Universal constant C
    threshold = C * math.sqrt(n / d**3)

    # Generate a random max-CUT instance
    graph = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        graph[i][i] = 0

    # Compute the degree-d SOS moment matrix M
    M = [[0] * n for _ in range(n)]
    for i in range(n):
```

```

        for j in range(i+1, n):
            if graph[i][j] == 1:
                for k in range(d):
                    M[i][k] += Fraction(1, d)
                    M[j][k] += Fraction(1, d)

# Calculate the eigenvalue spectrum of M
eigenvalues = []
A = [row[:] for row in M]
for i in range(n):
    pivot = A[i][i]
    if pivot == 0:
        return {"metric_name": "eigenvalue", "metric_value": None, "instances_tested": 1, "conjecture_holds": False}
    for j in range(n):
        A[j][i] /= pivot
    for k in range(n):
        if k != i:
            factor = A[k][i]
            for j in range(n):
                A[k][j] -= factor * A[i][j]
    eigenvalues.append(A[i][i])

# Verify if eigenvalues exceed the threshold
max_eigenvalue = max(eigenvalues)
if max_eigenvalue > threshold:
    return {"metric_name": "eigenvalue", "metric_value": max_eigenvalue, "instances_tested": 1, "conjecture_holds": False}
else:
    return {"metric_name": "eigenvalue", "metric_value": max_eigenvalue, "instances_tested": 1, "conjecture_holds": True}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"Max eigenvalue exceeds threshold\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
tances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'eigenvalue', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds':
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db715b38.py", line 98, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db715b38.py", line 98, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with KeyError: 'seed' before producing valid results | next: Re-run test with detailed logging to diagnose 'mapping_undefined' error

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	47851
2	preregistration	ollama_remote	qwen3:8b	0	0	27588
3	novelty	ollama_remote	qwen3:8b	0	0	24022
4	novelty	ollama_remote	qwen3:8b	0	0	15783
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13382
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9509
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11669
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11445
9	judge	ollama_remote	qwen3:8b	0	0	18327

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 179577 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/03dda0d662e2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/03dda0d662e2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/03dda0d662e2.tar.gz` (if generated)